

CRIANDO TRABALHOS ACADÊMICOS COM SPECCOMPILER: GUIA PRÁTICO DO TEMPLATE ABNT

SpecCompiler Team

Documento de exemplo do template ABNT.

Orientador: Este documento é auto-demonstrativo

Online

2025

SpecCompiler Team

**CRIANDO TRABALHOS ACADÊMICOS COM SPECCOMPILER:
GUIA PRÁTICO DO TEMPLATE ABNT**

Documento de exemplo do template ABNT.

Orientador: Este documento é auto-demonstrativo

Online

2025

Ficha catalográfica
Elaborada conforme os dados bibliográficos da publicação. Substitua esta página pela ficha catalográfica definitiva fornecida pela biblioteca responsável. CDD / CDU

FOLHA DE APROVAÇÃO

Documento destinado ao registro da aprovação pela banca examinadora. Substitua esta página pela folha de aprovação definitiva emitida ou validada pela instituição.

Autor(a) _____

Título _____

Data de aprovação _____

Membro da banca examinadora

Membro da banca examinadora

Membro da banca examinadora

*Este documento é dedicado a todos que
buscam uma forma mais simples de criar
trabalhos acadêmicos sem abrir mão da
conformidade com as normas ABNT.*

AGRADECIMENTOS

Agradecemos à comunidade de software livre, especialmente aos desenvolvedores do Pandoc e SQLite, que tornaram possível a criação do SpecCompiler.

Simplicidade é a sofisticação máxima.

— Leonardo da Vinci

RESUMO

Este documento serve como tutorial e demonstração do template Associação Brasileira de Normas Técnicas (ABNT) do SpecCompiler. Através de uma estrutura auto-referencial, apresenta-se como criar trabalhos acadêmicos em formato Markdown que são automaticamente convertidos para documentos DOCX em conformidade com a NBR 14724 ([NBR 14724, 2011](#)). São demonstradas as funcionalidades de elementos pré-textuais, textuais e pós-textuais, além de recursos como tabelas, quadros, gráficos baseados em dados, figuras, citações bibliográficas e listas automáticas de siglas, figuras e tabelas.

Palavras-chave: SpecCompiler, ABNT, markdown, trabalhos acadêmicos, NBR 14724.

ABSTRACT

This document serves as a tutorial and demonstration of the SpecCompiler ABNT template. Through a self-referential structure, it presents how to create academic works in Markdown format that are automatically converted to DOCX documents in compliance with NBR 14724. The functionalities of pre-textual, textual and post-textual elements are demonstrated, as well as features such as tables, frames, data-driven charts, figures, bibliographic citations and automatic lists of abbreviations, figures and tables.

LISTA DE FIGURAS

Figura 1 – Capivara descansando - exemplo de figura com legenda.....	16
Figura 2 – Capivara com altura limitada para composição da página.....	19
Figura 3 – Fluxo de criação de documento acadêmico.....	21
Figura 4 – Capivara — exemplo de figura com fonte por citação.....	22

LISTA DE TABELAS

Tabela 1 – Sintaxe do atributo source.....	21
Tabela 2 – Elementos pré-textuais conforme NBR 14724.....	23
Tabela 3 – Dados regionais — exemplo de tabela CSV.....	24
Tabela 4 – Comparação entre AsciiMath e LaTeX.....	25

LISTA DE ABREVIATURAS E SIGLAS

ABNT	Associação Brasileira de Normas Técnicas
APA	American Psychological Association
IBGE	Instituto Brasileiro de Geografia e Estatística
IEEE	Institute of Electrical and Electronics Engineers
OOXML	Office Open XML
specc	SpecCompiler
UML	Unified Modeling Language

SUMÁRIO

AGRADECIMENTOS.....	6
RESUMO.....	8
ABSTRACT.....	9
LISTA DE FIGURAS.....	10
LISTA DE TABELAS.....	11
LISTA DE ABREVIATURAS E SIGLAS.....	12
1 Introdução.....	15
1.1 Por que SpecCompiler?.....	16
1.2 Tipagem Forte vs Tipagem Fraca.....	17
1.3 Extensões ao Markdown.....	17
1.4 Estrutura deste Documento.....	18
2 Figuras e Ilustrações.....	19
2.1 Sintaxe Básica.....	19
2.2 Gráficos.....	20
2.3 Diagramas PlantUML.....	20
2.4 Atributos Comuns para Elementos Visuais.....	21
2.4.1 Dimensões (width e height).....	21
2.4.2 Atributo source.....	21
3 Tabelas e Quadros.....	23
3.1 A Diferença.....	23
3.2 Sintaxe list-table.....	23
3.3 Sintaxe CSV.....	24
3.4 Quadros e Listagens de Código.....	24
3.4.1 Quadro Textual (sem realce).....	25
3.4.2 Listagem de Código (com realce).....	25
3.5 Equações e Fórmulas com AsciiMath.....	25
3.5.1 Sintaxe Inline.....	26
3.5.2 Sintaxe em Bloco (Equação Numerada).....	26
3.5.3 Referência Rápida AsciiMath.....	27
4 Citações e Referências Bibliográficas.....	28
4.1 Tipos de Citação.....	28
4.2 Múltiplas Citações.....	28

4.3 Configuração: Citeproc e CSL.....	28
5 Referências Cruzadas.....	30
6 Siglas e Abreviaturas.....	31
7 Peculiaridades das Normas ABNT.....	32
7.1 Numeração de Páginas.....	32
7.2 Capítulos em Páginas Ímpares.....	32
7.3 Margens Espelhadas.....	32
7.4 Formatação de Tabelas (Padrão IBGE).....	32
7.5 Template Extensível.....	32
8 Conclusão.....	34
8.1 Uma Mudança de Paradigma.....	34
8.2 Documentação e Relatórios: A Linha que Desaparece.....	35
9 Trabalhos Futuros.....	36
9.1 Editor Colaborativo em Tempo Real.....	36
9.2 Interoperabilidade DOCX (Round-Trip).....	36
REFERENCIAS.....	37
APÊNDICE A – Referência Rápida de Sintaxe.....	38
A.1 Atributos do Documento.....	38
A.2 Figuras.....	38
A.3 Diagramas PlantUML.....	38
A.4 Gráficos com Query.....	38
A.5 Atributos de Dimensões e Fonte.....	38
A.6 Tabelas (list-table).....	38
A.7 Tabelas (CSV).....	38
A.8 Quadros.....	39
A.9 Citações.....	39
A.10 Notas de Rodapé.....	39
A.11 Referências Cruzadas.....	39
A.12 Siglas.....	39
A.13 Inclusão de Arquivos.....	39
A.14 Configuração (project.yaml).....	39
ANEXO A – Normas ABNT Consultadas.....	41

1 INTRODUÇÃO

O Markdown conquistou a escrita acadêmica por sua simplicidade. Uma sintaxe minimalista que permite ao autor concentrar-se no conteúdo, não na formatação. Ferramentas como Limarka (Vieira, 2016), Quarto (Posit PBC, 2022) e RMarkdown prometem o melhor dos dois mundos—escrever em Markdown, obter documentos academicamente formatados. Porém, há um paradoxo fundamental, todas essas ferramentas dependem do LaTeX como backend para geração de PDF.

Essa dependência cria uma **abstração vazada**. Quando o documento requer formatação específica —margens ABNT, capas institucionais, fichas catalográficas— o autor é frequentemente forçado a injetar código LaTeX diretamente no Markdown, quebrando a promessa de separação entre conteúdo e apresentação. A necessidade de instalar uma distribuição TeX completa (aproximadamente 4GB) e eventualmente depurar erros crípticos de compilação LaTeX contradiz a simplicidade prometida pelo Markdown.

Há ainda o problema do formato de saída. LaTeX é amplamente utilizado na engenharia acadêmica — e praticamente só lá. Periódicos científicos (de outras áreas) normalmente aceitam submissões em Word. Escritórios de transferência tecnológica trabalham com Word. Equipes de P&D em indústria colaboram em Word. O documento que realmente circula, é quase sempre um arquivo `.docx`. LaTeX produz PDFs elegantes, mas PDFs são destinos finais — não documentos de trabalho.

O SpecCompiler (specc) propõe uma abordagem diferente: **eliminar completamente o LaTeX da cadeia de produção**. O pipeline Markdown → specc → Office Open XML (OOXML) gera documentos DOCX nativos, editáveis, e em conformidade com as normas ABNT. Similar em objetivo e resultados ao (ABNabnTeX) (Araujo, 2015), mas fundamentalmente diferente em implementação.

O SpecCompiler foi projetado originalmente para **engenharia de requisitos** — rastrear requisitos, casos de verificação e decisões de design em projetos de software crítico. Porém, ao tratar normas de publicação científica (como ABNT NBR 14724) como **especificações formais**, o sistema se adapta naturalmente à escrita acadêmica. O mesmo motor que valida rastreabilidade de requisitos valida a estrutura de uma monografia.

Essa convergência reflete uma verdade mais profunda: **documentação e relatórios são faces da mesma moeda**. Uma especificação de requisitos de software e uma dissertação de

mestrado compartilham a necessidade de estrutura verificável, referências cruzadas consistentes, e conformidade com normas.

Este documento demonstra, de forma prática, como utilizar o SpecCompiler para criar uma monografia. Cada seção é um exemplo vivo das funcionalidades disponíveis, conforme ilustra a [Figura 1](#).

Figura 1 – Capivara descansando - exemplo de figura com legenda



Fonte: Autor

1.1 Por que SpecCompiler?

O ecossistema de ferramentas Markdown para trabalhos acadêmicos—Limarka ([Vieira, 2016](#)), Quarto ([Posit PBC, 2022](#)), RMarkdown—compartilha uma característica comum, todas utilizam LaTeX como backend para geração de PDF. Isso significa que, apesar da simplicidade do Markdown na superfície, o autor ainda precisa de uma distribuição TeX completa instalada, e eventualmente precisará depurar erros de LaTeX quando a abstração “vazar”.

O SpecCompiler adota uma abordagem diferente, não há LaTeX no pipeline. O [Quadro 1](#) ilustra a diferença de sintaxe entre LaTeX e Markdown:

Quadro 1 – Comparação de sintaxe: LaTeX vs Markdown

LATEX	MARKDOWN
<code>\section{Introdução}</code>	<code>## Introdução</code>
<code>\textbf{texto em negrito}</code>	<code>**texto em negrito**</code>
<code>\textit{texto em itálico}</code>	<code>*texto em itálico*</code>
<code>\begin{itemize}</code>	
<code>\item item 1</code>	<code>- item 1</code>
<code>\item item 2</code>	<code>- item 2</code>
<code>\item item 3</code>	<code>- item 3</code>
<code>\end{itemize}</code>	


```

\begin{figure}[htbp]
  \centering
  \includegraphics{imagem.jpg}
  \caption{Legenda}
\end{figure}
\cite{autor2024}
\fig{id{caption="Legenda"}}
imagem.jpg
[autor2024] (@cite)

```

Fonte: Elaboração própria

As principais funcionalidades do SpecCompiler incluem:

1. **Sem dependência LaTeX:** O pipeline utiliza exclusivamente Pandoc e OOXML—nenhuma distribuição TeX necessária
2. **Saída editável e colaborativa:** Diferentemente do PDF (formato de visualização), o DOCX permite controle de alterações, comentários em linha, e revisão por orientadores usando ferramentas familiares (Word, LibreOffice, Google Docs)
3. **Conformidade automática:** Pós-processamento OOXML aplica formatação ABNT (bordas de tabela IBGE, margens, numeração) conforme NBR 14724 ([NBR 14724, 2011](#))
4. **Campos nativos:** Sumário, lista de figuras e tabelas usam campos do Word (atualização automática ao abrir o documento)
5. **Banco de dados:** Estrutura do documento em SQLite permite validação semântica e consultas para gráficos
6. **Citações bibliográficas:** Integração com Citeproc para referências no padrão ABNT ([NBR 6023, 2018](#))

1.2 Tipagem Forte vs Tipagem Fraca

No LaTeX, `\section{Introdução}` é um comando de formatação—nada impede `\section{Conclusão}` aparecer antes de `\section{Introdução}`. No Markdown puro, `## Introdução` é ainda mais genérico: um cabeçalho sem semântica alguma.

No SpecCompiler, quando você escreve `## Introdução`, o sistema:

1. Reconhece o texto “Introdução” como **alias implícito** do tipo `INTRODUCTION`
2. Verifica que `INTRODUCTION` é **permitido** em especificações `MONOGRAFIA`
3. Valida que aparece **após** elementos pré-textuais
4. Garante que há **exatamente uma** introdução (`max_count: 1`)

Cada tipo ABNT é um módulo Lua em `types/objects/`, carregado no build e compilado para o Spec-IR (SQLite). Por exemplo, o tipo `ABSTRACT` em `types/objects/abstract.lua` define:

- `implicit_aliases`: [“Resumo”, “Abstract”, “Resume”, “Resumen”]
 - `extends`: “PRE_TEXTUAL”
 - `attributes`: [{ name: “keywords”, type: “STRING” }]
- Especificações bem tipadas não falham.

1.3 Extensões ao Markdown

O Markdown original é deliberadamente minimalista—e essa simplicidade tem um custo. A especificação original não contempla tabelas, fórmulas matemáticas, referências cruzadas

numeradas, nem figuras com legendas e fontes. Para escrita acadêmica, essas lacunas são críticas.

O SpecCompiler estende o Markdown com sintaxe adicional para suprir essas necessidades:

- **Tabelas:** Sintaxes `list-table:` e `csv:` para tabelas complexas com cabeçalhos e alinhamento
 - **Matemática:** Notação AsciiMath (mais legível que LaTeX) para equações inline e em bloco
 - **Figuras numeradas:** Blocos `fig:` com legendas, fontes e numeração automática
 - **Referências cruzadas:** Links `[type:identificador](#)` que se resolvem para “Figura 1”, “Tabela 2”, etc.
 - **Quadros e listagens:** Blocos `listing:` e `src.<ext>:` para código com moldura ABNT
 - **Gráficos:** Blocos `chart:` que consultam o banco de dados para gerar visualizações
 - **Diagramas:** Blocos `puml:` para diagramas UML via PlantUML
 - **Inclusão de arquivos:** Blocos `include` para dividir documentos longos em capítulos e apêndices separados
 - **Fechamento de seção:** Uma quebra temática `----` encerra a seção atual sem abrir um novo título, fazendo o conteúdo seguinte pertencer ao capítulo pai (o marcador é consumido e não vira linha horizontal)
 - **Notas de rodapé:** Sintaxe nativa do Pandoc¹ para notas de rodapé numeradas automaticamente
- As seções seguintes demonstram essas extensões em uso prático.

1.4 Estrutura deste Documento

Este tutorial segue a estrutura padrão de uma monografia conforme NBR 14724 ([NBR 14724, 2011](#)):

- **Elementos pré-textuais:** Capa, folha de rosto, resumo, listas, sumário
- **Elementos textuais:** Introdução, desenvolvimento, conclusão
- **Elementos pós-textuais:** Referências, apêndices, anexos

A quebra temática acima encerra a subseção *Estrutura deste Documento*: este parágrafo de fechamento pertence ao escopo do capítulo *Introdução*, e não à subseção anterior. Os capítulos seguintes demonstram cada extensão em uso prático.

¹ Notas de rodapé são suportadas nativamente via Pandoc `commonmark_x`. Esta é uma nota de exemplo que aparecerá no rodapé da página.

2 FIGURAS E ILUSTRAÇÕES

As figuras são inseridas utilizando blocos de código com o prefixo `fig:` seguido de um identificador único. Os atributos `caption` e `source` definem a legenda e a fonte da ilustração.

2.1 Sintaxe Básica

Para inserir uma figura:

```
```fig:identificador{caption="Legenda da figura" source="Fonte"}
caminho/para/imagem.jpg
```
```

A legenda é posicionada abaixo da figura, seguida da fonte — conforme exige a ABNT.

Quando a imagem original é alta demais para a página, limite a altura diretamente nos atributos do bloco. A largura será ajustada proporcionalmente pelo processador DOCX:

```
```fig:identificador-limitado{caption="Legenda da figura limitada" source="Fonte"
height="7cm"}
caminho/para/imagem.jpg
```
```

A [Figura 2](#) usa a mesma imagem do exemplo inicial, mas com altura máxima definida para deixar mais espaço para texto, legenda e fonte na página.

Figura 2 – Capivara com altura limitada para composição da página



Fonte: Autor

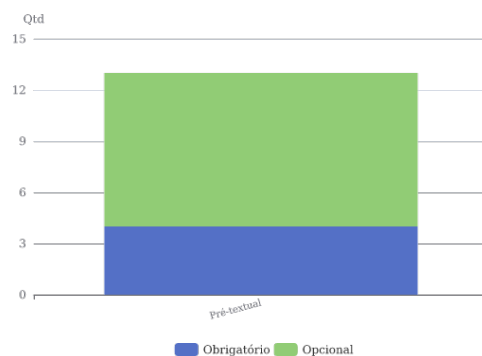
2.2 Gráficos

Gráficos são renderizados usando ECharts², uma biblioteca de visualização de dados. O corpo do bloco contém a configuração JSON do gráfico, e os dados podem vir de duas fontes:

- **query**: consulta uma view SQL do banco de dados do documento
- **generator**: invoca uma função Lua que gera os dados programaticamente

A [Gráfico 1](#) demonstra o uso de `query` — os dados vêm diretamente do próprio documento, mostrando a distribuição dos tipos ABNT utilizados nesta monografia:

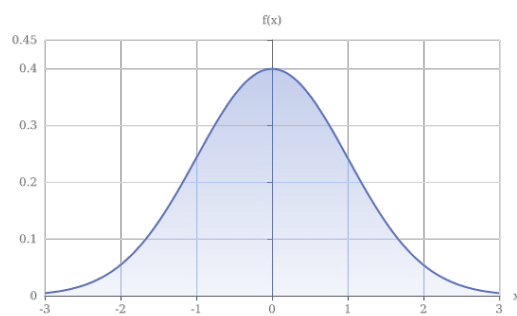
Gráfico 1 – Tipos ABNT utilizados neste documento



Fonte: O autor

Para o template ABNT, geradores Lua são particularmente úteis para visualizar funções matemáticas — como a [Gráfico 2](#), que renderiza a distribuição normal a partir de parâmetros:

Gráfico 2 – Distribuição Normal — gerada via Lua



Fonte: O autor

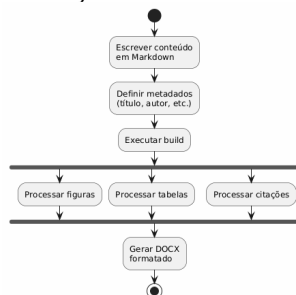
2.3 Diagramas PlantUML

Diagramas Unified Modeling Language (UML) podem ser gerados a partir de código PlantUML. O sistema processa os diagramas em paralelo para melhor desempenho.

² Apache ECharts — biblioteca open-source para visualização interativa. Documentação em <https://echarts.apache.org>.

A [Figura 3](#) ilustra o fluxo básico de criação de um documento:

Figura 3 – Fluxo de criação de documento acadêmico



Fonte: O autor

2.4 Atributos Comuns para Elementos Visuais

Os elementos `fig:`, `puml:` e `chart:` suportam atributos adicionais para controle de dimensões e fonte.

2.4.1 Dimensões (width e height)

Para controlar o tamanho dos elementos visuais:

```

```fig:exemplo{caption="Figura redimensionada" width="300" height="200"}
imagem.jpg
```

```

- Valores numéricos são interpretados como pixels: `width="400"` → 400px
- Unidades CSS são suportadas: `width="50%"`, `width="10cm"`, `height="3in"`
- Se apenas um atributo for informado, o outro é ajustado proporcionalmente (mantendo aspect ratio)
- Para gráficos (`chart:`), os valores padrão são 600×400 pixels
- Ambos os atributos são opcionais

2.4.2 Atributo source

O atributo `source` define a fonte da ilustração conforme ABNT:

Tabela 1 – Sintaxe do atributo source

| Sintaxe | Resultado |
|----------------------------------|------------------------------------|
| <code>source="Autor"</code> | Fonte: Autor |
| <code>source="@silva2024"</code> | Fonte: (SILVA 2024) — via citeproc |
| (omitido) | Fonte: Elaborado pelo autor |

Fonte: O autor

- **Texto literal:** Renderiza como “Fonte: {texto}”

- **Citação BibTeX:** `source="@chave"` integra com citeproc, usando a formatação do arquivo CSL
- **Omitido:** Aplica automaticamente “Fonte: Elaborado pelo autor” (padrão ABNT)
Exemplo com citação bibliográfica:

Figura 4 – Capivara — exemplo de figura com fonte por citação



Fonte: IBGE ([1993](#))

3 TABELAS E QUADROS

A ABNT distingue entre **tabelas** e **quadros** — uma distinção importante que o SpecCompiler respeita.

3.1 A Diferença

Conforme as normas de apresentação tabular do Instituto Brasileiro de Geografia e Estatística (IBGE) (IBGE, 1993):

- **Tabela:** Apresenta dados numéricos/estatísticos. Formatação com bordas abertas nas laterais (sem linhas verticais nas extremidades).
- **Quadro:** Apresenta informações textuais organizadas. Formatação com bordas fechadas em todos os lados.

O [Quadro 2](#) resume estas diferenças:

Quadro 2 – Diferença entre Tabela e Quadro conforme ABNT/IBGE

| |
|--|
| TABELA |
| - Contém dados numéricos/estatísticos |
| - Bordas abertas nas laterais |
| - Linhas horizontais: topo, separador do cabeçalho, base |
| - Sem linhas verticais nas extremidades |
| QUADRO |
| - Contém informações textuais |
| - Bordas fechadas em todos os lados |
| - Grade completa (todas as células delimitadas) |
| - Usado para definições, comparações textuais |

Fonte: IBGE (1993)

3.2 Sintaxe list-table

A sintaxe `list-table` é ideal para tabelas complexas. A [Tabela 2](#) demonstra uma tabela com múltiplas colunas:

Tabela 2 – Elementos pré-textuais conforme NBR 14724

| Elemento | Obrigatório | Descrição |
|---------------------|-------------|------------------------------|
| Capa | Sim | Identificação do trabalho |
| Folha de rosto | Sim | Dados essenciais do trabalho |
| Ficha catalográfica | Não | Verso da folha de rosto |
| Errata | Não | Lista de correções |
| Folha de aprovação | Sim* | Assinaturas da banca |
| Dedicatória | Não | Homenagem a pessoas |

| Elemento | Obrigatório | Descrição |
|------------------------------|-------------|--------------------------------------|
| Agradecimentos | Não | Reconhecimento a contribuições |
| Epígrafe | Não | Citação relacionada ao tema |
| Resumo em português | Sim | Síntese do trabalho (NBR 6028, 2003) |
| Resumo em língua estrangeira | Sim | Abstract |
| Lista de ilustrações | Não | Índice de figuras |
| Lista de tabelas | Não | Índice de tabelas |
| Lista de abreviaturas | Não | Glossário de siglas |
| Sumário | Sim | Índice de conteúdo (NBR 6027, 2012) |

Fonte: NBR 14724 (2011)

3.3 Sintaxe CSV

Para dados tabulares simples, a sintaxe CSV é mais concisa. A [Tabela 3](#) demonstra:

Tabela 3 – Dados regionais — exemplo de tabela CSV

| Região | 2022 | 2023 | 2024 |
|--------------|------|------|------|
| Norte | 150 | 175 | 200 |
| Nordeste | 280 | 310 | 350 |
| Centro-Oeste | 120 | 140 | 160 |
| Sudeste | 450 | 520 | 580 |
| Sul | 200 | 230 | 260 |

Fonte: Dados fictícios

3.4 Quadros e Listagens de Código

O SpecCompiler oferece duas sintaxes para criar quadros (código/listagens):

1. `listing:label` — Quadro genérico, sem realce de sintaxe. Ideal para texto estruturado, pseudocódigo ou informações textuais.
2. `src.<ext>:label` — Listagem com realce de sintaxe. O `<ext>` indica a linguagem (lua, c, python, js, etc.). Utiliza o Pandoc Skylighting para colorização.

Ambas as sintaxes suportam os atributos `caption` e `source`, e são renderizadas com moldura (borda) no template ABNT.

3.4.1 Quadro Textual (sem realce)

O [Quadro 3](#) demonstra a sintaxe `listing`: para informações textuais:

Quadro 3 – Principais vantagens do SpecCompiler

| |
|---|
| 1. SINTAXE SIMPLES |
| Markdown é intuitivo e legível mesmo em formato texto puro. |
| Não requer conhecimento de LaTeX ou linguagens complexas. |
| 2. CONFORMIDADE AUTOMÁTICA |
| O template ABNT cuida de margens, fontes, espaçamentos. |
| Validação estrutural impede documentos malformados. |
| 3. EXTENSIBILIDADE |
| Sistema de templates permite criar novos formatos. |
| O template ABNT estende o template padrão (default). |
| 4. INTEGRAÇÃO COM DADOS |
| Gráficos podem consultar o banco de dados. |
| Views SQL alimentam visualizações automaticamente. |

Fonte: Elaboração própria

3.4.2 Listagem de Código (com realce)

O [Quadro 4](#) demonstra a sintaxe `src.c`: para código-fonte com realce de sintaxe:

Quadro 4 – Programa Hello World em C

| |
|---|
| <code>#include <stdio.h></code> |
| <code>int main(void) {</code> |
| <code>printf("Hello, World!\n");</code> |
| <code>return 0;</code> |
| <code>}</code> |

Fonte: Elaboração própria

Outras linguagens suportadas incluem: `src.lua`:, `src.python`:, `src.java`:, `src.js`:, `src.sql`:, entre outras. A lista completa depende do Pandoc Skylighting.

3.5 Equações e Fórmulas com AsciiMath

O SpecCompiler utiliza AsciiMath para expressões matemáticas — uma sintaxe mais simples e legível que LaTeX. A [Tabela 4](#) compara as duas notações:

Tabela 4 – Comparação entre AsciiMath e LaTeX

| Expressão | AsciiMath | LaTeX |
|-----------|------------------|--------------------------|
| Fração | <code>x/y</code> | <code>\frac{x}{y}</code> |

| Expressão | AsciiMath | LaTeX |
|--------------------|------------------------------------|--------------------------------------|
| Raiz quadrada | $\text{sqrt}(x)$ | $\sqrt{x}$ |
| Potência | x^2 | $x^{\{2\}}$ |
| Subscrito | x_i | $x_{\{i\}}$ |
| Somatório | $\text{sum}_{(i=1)}^n i$ | $\sum_{i=1}^n i$ |
| Integral | $\text{int}_{0^1} f(x) \, dx$ | $\int_0^1 f(x) \, dx$ |
| Fórmula quadrática | $(-b \pm \sqrt{b^2 - 4ac}) / (2a)$ | $\frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$ |

Fonte: Elaboração própria

3.5.1 Sintaxe Inline

Para inserir matemática no meio do texto, use a sintaxe `\math: expressão`. Exemplos:

- O teorema de Pitágoras: $a^2 + b^2 = c^2$
- Área do círculo: $A = \pi r^2$
- Somatório: $\sum_{i=1}^n i = \frac{n(n+1)}{2}$

3.5.2 Sintaxe em Bloco (Equação Numerada)

Para equações destacadas e numeradas, use:

$$a^2 + b^2 = c^2 \quad (1)$$

A [Equação 1](#) demonstra o teorema fundamental da geometria euclidiana.

A fórmula quadrática de Bhaskara ([Equação 2](#)) fornece as raízes de equações do segundo grau:

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a} \quad (2)$$

A integral de Gauss ([Equação 3](#)) é fundamental na teoria das probabilidades e estatística:

$$\int_{-\infty}^{\infty} e^{-x^2} dx = \sqrt{\pi} \quad (3)$$

A entropia de Shannon ([Equação 4](#)) quantifica a informação média em uma fonte de dados:

$$H(X) = - \sum_{i=1}^n p(x_i) \log_2 p(x_i) \quad (4)$$

As equações demonstram a capacidade do SpecCompiler de renderizar fórmulas matemáticas complexas usando notação AsciiMath, que é mais intuitiva que LaTeX para a maioria dos casos.

3.5.3 Referência Rápida AsciiMath

- Operações básicas: `+`, `-`, `*`, `/`, `=`, `!=`, `<`, `>`, `<=`, `>=`
- Símbolos gregos: `alpha`, `beta`, `gamma`, `delta`, `pi`, `theta`, `omega`
- Funções: `sin`, `cos`, `tan`, `log`, `ln`, `exp`, `sqrt`
- Agrupamento: parênteses `()`, colchetes `[]`, chaves `{}`
- Matrizes: `[[a,b],[c,d]]` para matriz 2x2

4 CITAÇÕES E REFERÊNCIAS BIBLIOGRÁFICAS

O SpecCompiler utiliza Citeproc para processar citações—não o tradicional toolchain BibTeX/Biber do LaTeX. Essa escolha simplifica significativamente o fluxo de trabalho: um único arquivo `.bib` e um estilo CSL são suficientes, sem necessidade de múltiplas compilações ou arquivos auxiliares (`.aux`, `.bbl`, `.blg`, `.bcf`, `.run.xml`). O processamento segue as normas NBR 10520 (NBR 10520, 2002) e NBR 6023 (NBR 6023, 2018).

4.1 Tipos de Citação

Citação indireta (entre parênteses): A referência aparece ao final da frase.

A formatação de trabalhos acadêmicos deve seguir normas padronizadas (NBR 14724, 2011).

Citação direta (no texto): O autor faz parte da sentença.

Segundo NBR 14724 (2011), os trabalhos acadêmicos devem conter elementos pré-textuais, textuais e pós-textuais.

4.2 Múltiplas Citações

Para citar múltiplas obras simultaneamente, separe as chaves com ponto e vírgula:

Diversos autores tratam da normalização de documentos acadêmicos (NBR 10520, 2002; NBR 14724, 2011; NBR 6023, 2018).

4.3 Configuração: Citeproc e CSL

O processamento de citações utiliza dois componentes:

1. **Citeproc**: Processador de citações integrado ao Pandoc que converte `[chave](@cite)` em citações formatadas
2. **CSL (Citation Style Language)**: Arquivo XML que define o estilo de formatação das citações

O template ABNT já fornece o estilo de citação (`abnt.csl`, declarado em `models/abnt/config.lua`); o `project.yaml` precisa apenas indicar a bibliografia:

```
# Configuração de bibliografia (citeproc)
bibliography: references.bib
# csl: opcional – só declare aqui para sobrescrever o estilo do modelo
```

O arquivo CSL determina:

- Formato da citação no texto: (Autor, ano) vs [Autor, ano]
- Formato das entradas na bibliografia
- Ordenação das referências
- Regras de abreviação (et al.)

O estilo ABNT segue a NBR 10520 ([NBR 10520, 2002](#)) para citações e NBR 6023 ([NBR 6023, 2018](#)) para referências.

5 REFERÊNCIAS CRUZADAS

O SpecCompiler suporta referências cruzadas para figuras, tabelas, quadros e seções:

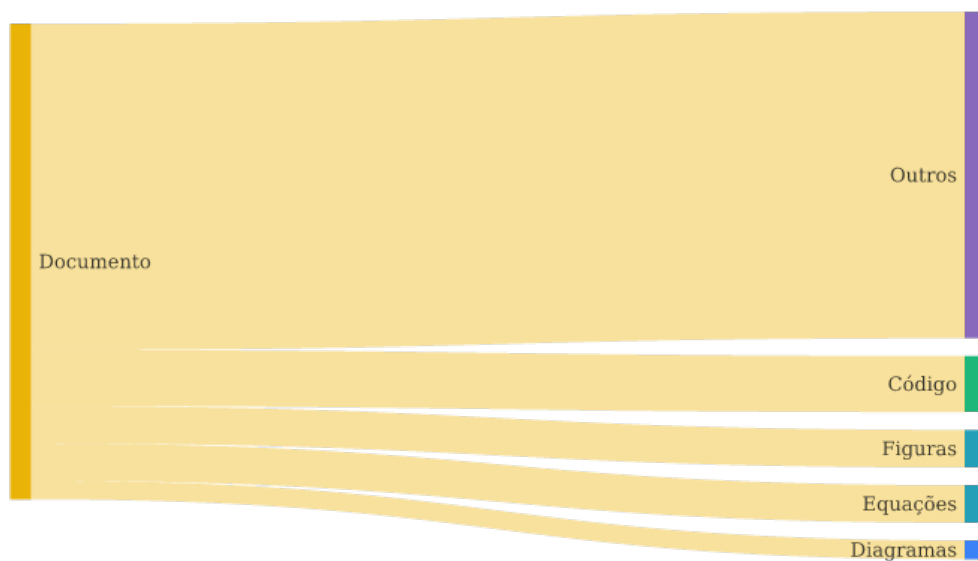
- Figuras: conforme demonstrado na [Figura 1](#)
- Tabelas: os dados da [Tabela 3](#) mostram...
- Quadros: o [Quadro 2](#) explica...

A sintaxe usa `[type:identificador] (#)`:

```
Figura \[fig:id] (#)
Tabela \[csv:id] (#)
Quadro \[listing:id] (#)
```

O sistema infere o **tipo de relação** (XREF_FIGURE, XREF_TABLE, XREF_LISTING, XREF_MATH) para cada elemento flutuante durante a compilação. O gráfico a seguir ([Gráfico 3](#)) mostra a distribuição desses tipos por seção—dados extraídos diretamente do banco de dados do documento:

Gráfico 3 – Tipos XREF por Seção — inferência de relações em tempo de compilação
Seções → Tipos de Referência



Fonte: O autor

6 SIGLAS E ABREVIATURAS

O SpecCompiler gerencia siglas automaticamente. Na primeira ocorrência, o termo completo é exibido com a sigla entre parênteses. Nas ocorrências seguintes, apenas a sigla aparece.

Para definir uma sigla, use:

```
`sigla: Termo Completo (SIGLA)`
```

Ao longo deste documento, várias siglas foram utilizadas: ABNT, SDN, ECH, UML, IBGE. Todas aparecem automaticamente na Lista de Abreviaturas e Siglas.

7 PECULIARIDADES DAS NORMAS ABNT

As normas ABNT para trabalhos acadêmicos incluem requisitos complexos que o SpecCompiler implementa automaticamente.

7.1 Numeração de Páginas

A NBR 14724 ([NBR 14724, 2011](#)) estabelece que os elementos pré-textuais são **contados, mas não numerados**. A numeração visível começa apenas na parte textual, em algarismos arábicos.

Curiosamente, o uso de algarismos romanos nos pré-textuais (i, ii, iii...) **não é exigido pela norma** — é uma adição de algumas instituições ([Equipe abnTeX2, 2015](#)). O SpecCompiler suporta ambas as configurações.

7.2 Capítulos em Páginas Ímpares

Para impressão frente e verso, a norma exige que capítulos iniciem em páginas ímpares (lado direito do documento aberto). O SpecCompiler insere automaticamente páginas em branco quando necessário.

7.3 Margens Espelhadas

Para encadernação, as margens são espelhadas:

- Margem interna (encadernação): 3 cm
- Margem externa: 2 cm

7.4 Formatação de Tabelas (Padrão IBGE)

Tabelas seguem o padrão do IBGE ([IBGE, 1993](#)):

- Bordas abertas nas laterais
- Apenas linhas horizontais (topo, separador, base)
- Título acima da tabela
- Fonte abaixo

7.5 Template Extensível

O template ABNT é apenas um exemplo das capacidades do SpecCompiler. O sistema de extensões permite criar templates para qualquer padrão documental — Institute of Electrical and Electronics Engineers (IEEE), American Psychological Association (APA), normas corporativas, etc.

O template ABNT estende o template `default`, herdando funcionalidades como gráficos, PlantUML e tabelas (list-table, CSV, TSV), e adicionando tipos específicos para a estrutura acadêmica brasileira.

8 CONCLUSÃO

Este documento demonstrou como utilizar o template ABNT do SpecCompiler para criar trabalhos acadêmicos em conformidade com a NBR 14724 ([NBR 14724, 2011](#)). Através de exemplos práticos, foram apresentados:

1. Inserção de figuras com legendas padronizadas ([Figura 1](#))
2. Gráficos com dados do banco ([Gráfico 1](#))
3. Diagramas PlantUML ([Figura 3](#))
4. Tabelas com sintaxe list-table ([Tabela 2](#))
5. Tabelas com sintaxe CSV ([Tabela 3](#))
6. Quadros para informações textuais ([Quadro 3](#))
7. Citações bibliográficas conforme ABNT
8. Referências cruzadas automáticas
9. Gerenciamento de siglas

8.1 Uma Mudança de Paradigma

Mais do que uma ferramenta de formatação, o SpecCompiler representa uma mudança de paradigma na produção de documentos acadêmicos brasileiros. Ao eliminar o LaTeX do pipeline de geração, rompe-se com décadas de dependência de um ecossistema notoriamente complexo.

A própria nomenclatura do ecossistema TeX revela sua fragmentação: TeX é o sistema de tipografia original de Donald Knuth (1978); LaTeX é uma camada de macros sobre o TeX criada por Leslie Lamport; pdfTeX, XeTeX e LuaTeX são diferentes *engines* que processam o código-fonte; e distribuições como TeX Live e MiKTeX empacotam milhares de pacotes com dependências cruzadas. Para o usuário iniciante, a distinção entre esses componentes raramente é clara—e quando algo falha, a depuração exige conhecimento de múltiplas camadas.

Plataformas como Overleaf ([Digital Science, 2024](#)) mitigaram parte dessa complexidade ao oferecer um ambiente web pré-configurado, democratizando o acesso ao LaTeX. Porém, ao custo de *lock-in*: o documento existe apenas na nuvem do fornecedor, e a colaboração depende de todos os participantes terem conta na plataforma.

O formato DOCX de saída do SpecCompiler não é uma limitação, mas uma característica estratégica. Diferentemente do PDF—um formato de visualização, não de edição—o DOCX permite colaboração nativa: controle de alterações, comentários em linha, e revisão por orientadores e bancas usando ferramentas já instaladas em seus computadores (Microsoft Word, LibreOffice, Google Docs). Não há necessidade de instalar software especializado ou criar contas em plataformas proprietárias.

O SpecCompiler automatiza a formatação e validação do documento, permitindo que o autor concentre-se no conteúdo.

8.2 Documentação e Relatórios: A Linha que Desaparece

Ferramentas tradicionais separam **documentação** (conteúdo estático, escrito manualmente) de **relatórios** (conteúdo dinâmico, gerado de dados). O SpecCompiler dissolve essa fronteira.

Neste documento, elementos como:

- a seção ## Sumário — sumário gerado automaticamente da estrutura
 - a seção ## Lista de Figuras — lista de figuras extraída do banco de dados
 - a seção ## Lista de Siglas — lista de siglas populada durante a compilação
 - `chart:abnt-types{query="..."}` — gráfico que consulta views SQL
- ...não são conteúdo estático. São **consultas materializadas** que se atualizam a cada

build. O documento é simultaneamente **documentação** (texto autoral) e **relatório** (dados do sistema de tipos).

Essa fusão é possível porque o SpecCompiler armazena tudo em SQLite: objetos, relações, atributos, hierarquias. Views SQL podem agregar, filtrar e projetar esses dados—e gráficos ECharts podem visualizá-los. O template apenas define **quais views existem e como renderizá-las**.

O mesmo mecanismo que gera “Lista de Figuras” numa monografia pode gerar “Matriz de Rastreabilidade” num SRS ou “Relatório de Cobertura” num TRR. A diferença é o template e as views—não o motor.

9 TRABALHOS FUTUROS

O desenvolvimento do SpecCompiler segue duas direções prioritárias:

9.1 Editor Colaborativo em Tempo Real

Integração com um editor web que permita múltiplos autores editarem o mesmo documento simultaneamente, similar à experiência do Google Docs ou Overleaf—mas com o documento-fonte em Markdown e armazenamento local ou em repositórios Git, evitando dependência de plataformas proprietárias.

9.2 Interoperabilidade DOCX (Round-Trip)

Atualmente, o fluxo é unidirecional: Markdown $\rightarrow$ DOCX. Um objetivo futuro é suportar importação de arquivos DOCX existentes, convertendo-os para a estrutura Markdown do SpecCompiler. Isso permitiria:

- Migrar trabalhos iniciados em Word para o ecossistema SpecCompiler
- Incorporar revisões feitas diretamente no DOCX de volta ao fonte Markdown
- Ciclos iterativos de edição (*round-trip*) sem perda de formatação semântica

Essa interoperabilidade bidirecional eliminaria a principal barreira de adoção: a necessidade de começar do zero em uma nova ferramenta.

REFERENCIAS

ARAUJO, Lauro César. **A classe abntex2: Modelo canônico de trabalhos acadêmicos brasileiros compatível com as normas ABNT NBR 14724:2011, ABNT NBR 6024:2012 e outras.** [S.l.]: Equipe abnTeX2, 2015.

DIGITAL SCIENCE. **Overleaf: Online LaTeX Editor.** Disponível em: <<https://www.overleaf.com>>. Acesso em: 15 dez. 2024.

EQUIPE ABNTEX2. **HowTo: Numeração romana nos pré-textuais.** Disponível em: <<https://github.com/abntex/abntex2/wiki/HowToPretextuaisRomanos>>. Acesso em: 15 dez. 2024.

IBGE. **Normas de apresentação tabular.** 3. ed. Rio de Janeiro: Centro de Documentação e Disseminação de Informações, 1993.

NBR 10520: Informação e documentação — Apresentação de citações em documentos. Rio de Janeiro: Associação Brasileira de Normas Técnicas, 2002. p. 7

NBR 14724: Informação e documentação — trabalhos acadêmicos — apresentação. Rio de Janeiro: Associação Brasileira de Normas Técnicas, 2011. p. 15

NBR 14724: Informação e documentação — trabalhos acadêmicos — apresentação. Rio de Janeiro: Associação Brasileira de Normas Técnicas, 2024. p. 12

NBR 6023: Informação e documentação — Referências — Elaboração. Rio de Janeiro: Associação Brasileira de Normas Técnicas, 2018. p. 74

NBR 6024: Informação e documentação — Numeração progressiva das seções de um documento. Rio de Janeiro: Associação Brasileira de Normas Técnicas, 2012. p. 4

NBR 6027: Informação e documentação — Sumário — Apresentação. Rio de Janeiro: Associação Brasileira de Normas Técnicas, 2012. p. 3

NBR 6028: Informação e documentação — Resumo — Apresentação. Rio de Janeiro: Associação Brasileira de Normas Técnicas, 2003. p. 2

POSIT PBC. **Quarto: An open-source scientific and technical publishing system.**, 2022. Disponível em: <<https://quarto.org>>

VIEIRA, Eduardo de Santana Medeiros. **Limarka: Markdown para trabalhos acadêmicos com ABNT.**, 2016. Disponível em: <<https://github.com/abntex/limarka>>

APÊNDICE A – Referência Rápida de Sintaxe

A.1 Atributos do Documento

```
> institution: Nome da Instituição
> author: Nome do Autor
> title: Título do Trabalho
```

A.2 Figuras

```
```fig:id{caption="Legenda" source="Fonte" width="400" height="300"}
caminho/imagem.jpg
```
```

A.3 Diagramas PlantUML

```
```puml:id{caption="Título do diagrama" width="600"}
@startuml
...
@enduml
```
```

A.4 Gráficos com Query

```
```chart:id{query="nome_da_view" caption="Título" width="800" height="500"}
{ ...configuração ECharts... }
```
```

A.5 Atributos de Dimensões e Fonte

Aplicáveis a `fig:`, `puml:`, `chart:`:

- `width`: Largura (px, %, cm, in)
- `height`: Altura (px, %, cm, in)
- `source`: Fonte — texto, @citação, ou omitido → “Elaborado pelo autor”

A.6 Tabelas (list-table)

```
```list-table:id{caption="Título" source="Fonte"}
> header-rows: 1
> aligns: l,c,r
* - Col1
 - Col2
* - Valor1
 - Valor2
```
```

A.7 Tabelas (CSV)

```
```csv:id{caption="Título" source="Fonte"}
Col1,Col2,Col3
Val1,Val2,Val3
```
```

A.8 Quadros

```
```listing:id{caption="Título" source="Fonte"}
Conteúdo textual do quadro
```
```

A.9 Citações

```
[chave](@cite)           - Citação entre parênteses
[chave](@citep)          - Citação no texto
[chave1;chave2](@cite)   - Múltiplas citações
```

A.10 Notas de Rodapé

Texto com nota[^id]. - Referência inline

[^id]: Conteúdo da nota de rodapé.

A.11 Referências Cruzadas

```
Figura [fig:id](#)
Tabela [table:id](#)
Quadro [listing:id](#)
```

A.12 Siglas

```
`sigla: Termo Completo (SIGLA)`
```

A.13 Inclusão de Arquivos

Use `include` para compor um documento a partir de vários arquivos Markdown. Os caminhos são relativos ao arquivo que contém a diretiva:

```
```include
chapters/introducao.md
chapters/figuras-ilustracoes.md
chapters/referencias.md
chapters/appendice-sintaxe.md
```
```

O conteúdo incluído participa da mesma estrutura lógica do documento: cabeçalhos, figuras, tabelas, citações, siglas e referências cruzadas são processados em conjunto.

A.14 Configuração (project.yaml)

```
# Template e tipos
template: abnt
type_file: ../dist/db/seed.sql

# Arquivos fonte
doc_files:
  - monografia.md

# Saída
output_dir: build/
output_formats: [docx]

# Estilos DOCX
docx:
```

```
preset: academico
reference_doc: ../dist/reference.docx

# Bibliografia (citeproc); o estilo CSL é fornecido pelo modelo ABNT.
bibliography: references.bib
```


ANEXO A – Normas ABNT Consultadas

Lista das normas da ABNT consultadas para desenvolvimento do template ([NBR 10520, 2002](#); [NBR 14724, 2011, 2024](#); [NBR 6023, 2018](#); [NBR 6024, 2012](#); [NBR 6027, 2012](#); [NBR 6028, 2003](#)):

- **NBR 14724:2011** — Trabalhos acadêmicos — Apresentação
- **NBR 14724:2024** — Trabalhos acadêmicos — Apresentação (atualização)
- **NBR 6023:2018** — Referências — Elaboração
- **NBR 6028:2003** — Resumo — Apresentação
- **NBR 6027:2012** — Sumário — Apresentação
- **NBR 6024:2012** — Numeração progressiva das seções
- **NBR 10520:2002** — Citações em documentos